

# Mobile App Update Impact and Failure Detection

1,137,987 Play Store reviews | 11 apps | 3 domains | collected by us, 1 April to 20 September 2026

## Team number: 10

|                        |                  |
|------------------------|------------------|
| Aditya Monish Kumar K  | CB.SC.U4CSE23103 |
| Regella Krishna Saketh | CB.SC.U4CSE23649 |
| Akshay KS              | CB.SC.U4CSE23104 |
| Harshini Vennela       | CB.SC.U4CSE23455 |
| Kanishka D             | CB.SC.U4CSE23155 |

Review 1 presentation: 20 marks

# Problem Statement and Research Gap

## Problem statement

Indian consumer apps get hundreds to over a thousand Play Store reviews every day. Many report real problems: a failed payment, a late delivery, a refund that never came, an app that crashes after an update.

A product team cannot read them all. Problems are noticed late, and it is hard to say which problem is biggest or whether things are getting better or worse.

1,539

reviews per day at the busiest app (Flipkart); 117 at the smallest

11 apps

in 3 domains: food and grocery, shopping, payments

## Research gap

- 1 Fixed set, no time view.** Reviews are studied as one fixed set; complaints are not followed over time.
- 2 Generic labels.** "Bug" and "feature request" do not match Indian delivery, shopping and payment problems (refunds, delivery delay, OTP, cancellation).
- 3 Tasks done alone.** Only classification or only sentiment; the results are not combined into something a manager can use.
- 4 Collection bias unchecked.** How reviews are collected can bias results, and this is rarely tested.

# Existing Methods, Proposed Approach and Objective

## Existing methods

**Pagano & Maalej (2013):** over 1M App Store reviews contain bug reports and feature requests.

**Chen et al. (2014), AR-Miner:** drops uninformative reviews, groups the rest by topic.

**Guzman & Maalej (2014):** feature extraction plus sentiment for each app feature.

**Maalej & Nabil (2015):** classifiers for bug, feature request, experience, rating.

**Hutto & Gilbert (2014), VADER:** rule-based sentiment for short informal text.

## Business objective

- Which problems hurt users most, in each app.
- Which new reviews need attention first.
- Whether a new app version made things worse.
- Whether this week's number of negative reviews is normal or unusual.

## Proposed approach: one pipeline from raw reviews to a problem-review classifier

### 1 Collect

1.20M reviews, 11 apps

### 2 Clean

1.14M reviews kept

### 3 Tag + sentiment

9 issues, VADER

### 4 Explore

app, issue, bias checks

### 5 Features

TF-IDF, SVD + 15

### 6 Classify

Logistic Regression,  
Random Forest

# Dataset: Source and Collection Method

## How we collected it

**Source:** Google Play Store, through the google-play-scraper package. No login.

**Own collection:** no Kaggle, UCI or GitHub dataset was used.

**Settings:** language English, country India, sort order NEWEST, 200 reviews per page.

**Window:** every review from 1 April to 20 September 2026, the same for all 11 apps (173 days).

**Apps:** Food & Grocery: Swiggy, Zomato, Blinkit, Domino's. Shopping: Myntra, Flipkart, Amazon, Meesho. Payments: Paytm, PhonePe, Google Pay.

**Why NEWEST:** MOST\_RELEVANT ran out of reviews (Swiggy at 11,200) and mixed years back to 2018; NEWEST has no ceiling.

**Documentation:** DATA\_SOURCES.md; one gzipped CSV per app in data/raw, data/clean and data/tagged.

1,202,729

raw reviews

1,137,987

after cleaning

11

apps in 3 domains

29

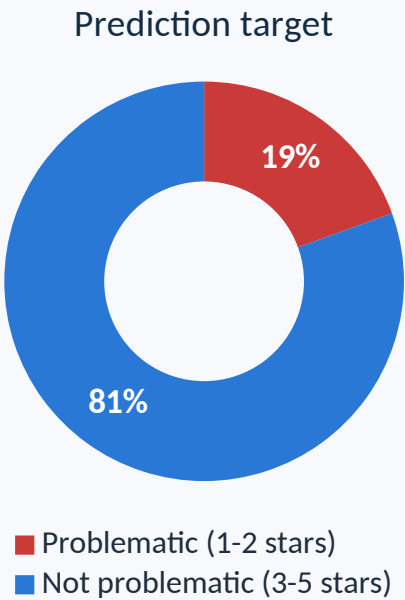
columns after tagging and sentiment

## Prediction target

is\_problematic = 1 if the rating is 1 or 2 stars:  
221,581 reviews (19.5%).

# Dataset: Key Variables and Target

| Variable                          | Type           | Meaning                              |
|-----------------------------------|----------------|--------------------------------------|
| app_name, domain                  | Category       | One of the 11 apps; its domain       |
| score                             | Integer 1 to 5 | Star rating given by the user        |
| content                           | Text           | The review text (the comment)        |
| thumbs_up                         | Integer        | Users who found the review helpful   |
| app_version                       | Text           | App version on the reviewer's phone  |
| review_date                       | Date and time  | When the review was posted           |
| review_length                     | Integer        | Number of words in the review        |
| issue_* (9 columns)               | 0 or 1         | Review mentions that issue category  |
| sentiment_neg, neu, pos, compound | Number         | VADER scores, compound from -1 to +1 |



**Target:** is\_problematic = 1 when score is 1 or 2.

**Counts:** 221,581 problematic (19.5%) and 916,406 not (80.5%).

**Consequence:** always saying "not problematic" is already 80.5% accurate, so accuracy alone is not enough.

# Dataset: Data Sample, Period and Limitations

| App      | Stars | Review text                                                                                            | Date       | Issue tag      | Sentiment |
|----------|-------|--------------------------------------------------------------------------------------------------------|------------|----------------|-----------|
| Swiggy   | 1     | ...assigning multiple orders for single delivery partner at the end all the orders are getting delayed | 2026-05-02 | delivery_delay | Negative  |
| Amazon   | 1     | i return my Order product... but didn't get any Refund even After waiting for 20 Days                  | 2026-07-14 | payment_refund | Neutral   |
| Flipkart | 5     | nice app                                                                                               | 2026-08-20 | none           | Positive  |
| PhonePe  | 5     | Yes my phone pe app not working                                                                        | 2026-05-20 | none           | Positive  |

## Collection period and limitations

### Very short reviews

Median 2 words; 66% have 3 words or fewer ("good", "nice app").

### Harsher than public rating

Written reviews sit 0.0 to 1.4 stars below each app's Play Store rating.

### Late-April feed gap

21 Apr to 5 May: positive reviews drop 59 to 90% for 5 apps. Rows kept and flagged.

### Missing values

app\_version empty for 13.4%; Zomato has no reviews 23 to 25 July (source gap).

# Cleaning: Missing Values, Duplicates and Outliers

## Missing values

**app\_version:** empty in 152,017 rows (13.4%). Kept: it is not a model feature and the text is still valid.

**content:** 3 empty texts set to "" (then removed by the short-text rule).

**All other columns:** no missing values.

## Duplicates

**By review\_id:** 0 duplicates.

**Repeated text:** 62% of reviews share their text with another ("good" appears 196,940 times). These are real users, so they are kept.

## Outliers (Tukey 1.5 x IQR fences)

| Field                 | Outside fences                          | Maximum | Decision                                                                 |
|-----------------------|-----------------------------------------|---------|--------------------------------------------------------------------------|
| thumbs_up             | 72,945 (6.4%); 93.6% have no vote       | 11,165  | Keep: real, heavily upvoted reviews. Use log scale, medians, rank tests. |
| review_length (words) | 167,193 (14.7%); fence is only 11 words | 167     | Keep: long reviews are the informative ones.                             |

**A check that caught a bug:** our first language rule counted emojis and wrongly removed about 76,000 short English reviews ("good" with a thumbs-up emoji). It now judges letters only.

# Cleaning: Encoding, Scaling and Notebook Evidence

## Encoding and scaling

**Text:** turned into numbers with TF-IDF (next section).

**Issue categories:** 9 columns of 0 or 1.

**Structural features:** 15 columns (9 flags, 4 sentiment scores, length, upvotes) scaled with StandardScaler: mean 0, spread 1.

**Derived fields:** month and review\_length (words).

## Before and after

**1.20M to 1.14M**

rows: raw to cleaned

**9 to 11 to 29**

columns: raw, cleaned, tagged

**Removed:** 64,742 rows (5.4%): 47,533 empty or under 3 characters, 7,420 with no letters, 9,789 non-English.

notebooks/Review\_1\_Analysis.ipynb · Section 3: Data cleaning (executed output)

```
In [7]: def latin_letter_ratio(s):
        letters = [c for c in s if c.isalpha()]
        return sum(c.isascii() for c in letters) / len(letters) if letters else 0.0

df = raw.copy()
df["content"] = df["content"].fillna("").astype(str)
steps = [("Raw scraped data", len(df))]

df = df.drop_duplicates(subset="review_id")
steps.append(("Duplicate review_id removed", len(df)))

df = df[df["content"].str.strip().str.len() >= 3]
steps.append(("Empty / < 3 characters removed", len(df)))

df = df[df["content"].map(lambda s: any(c.isalpha() for c in s))]
steps.append(("No letters (emoji/punctuation) removed", len(df)))

df = df[df["content"].map(latin_letter_ratio) >= 0.85]
steps.append(("Non-English (< 85% Latin letters) removed", len(df)))

df["month"] = df["review_date"].dt.to_period("M").astype(str)
df["review_length"] = df["content"].str.split().str.len()

before_after = pd.DataFrame(steps, columns=["step", "rows"])
before_after["removed"] = (-before_after["rows"].diff()).fillna(0).astype(int)
print(f"Columns: {raw.shape[1]} raw -> {df.shape[1]} after cleaning")
before_after
```

Out[7]: Columns: 9 raw -> 11 after cleaning

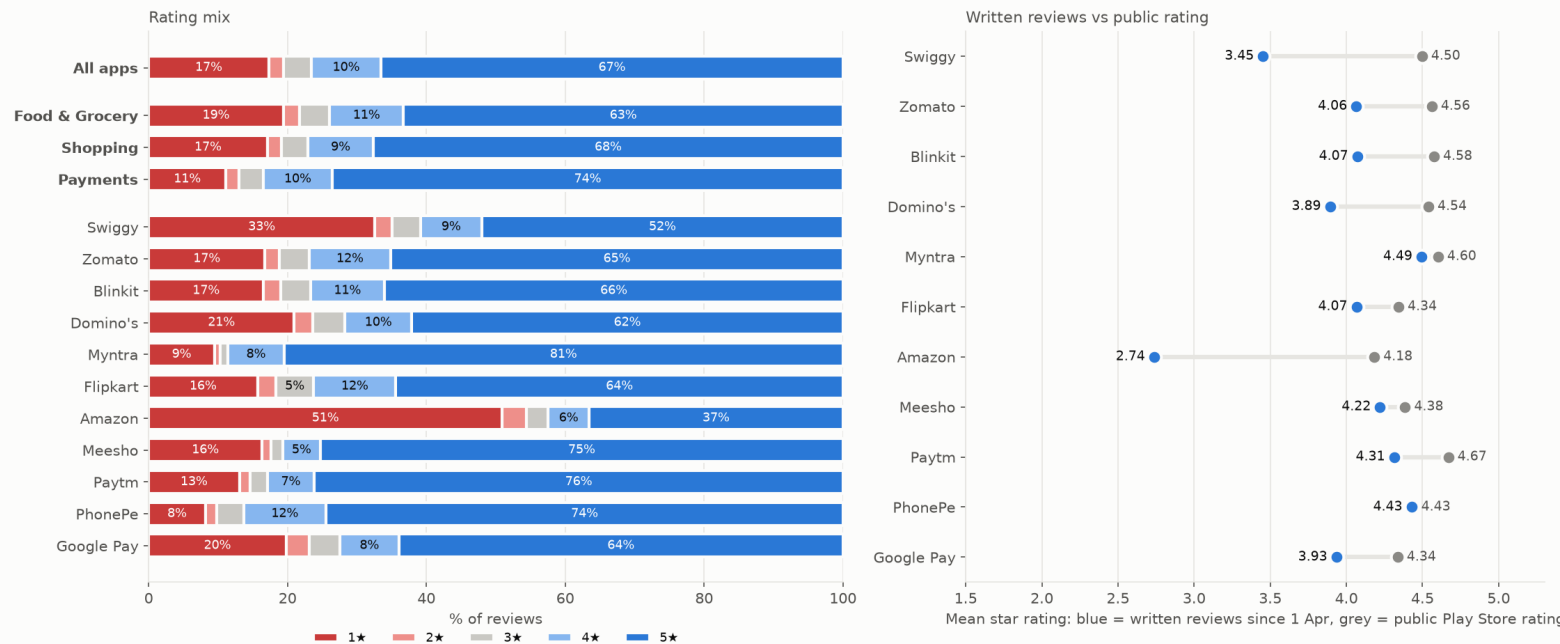
|   | step                                      | rows    | removed |
|---|-------------------------------------------|---------|---------|
| 0 | Raw scraped data                          | 1202729 | 0       |
| 1 | Duplicate review_id removed               | 1202729 | 0       |
| 2 | Empty / < 3 characters removed            | 1155196 | 47533   |
| 3 | No letters (emoji/punctuation) removed    | 1147776 | 7420    |
| 4 | Non-English (< 85% Latin letters) removed | 1137987 | 9789    |

Evidence: executed cell in notebooks/Review\_1\_Analysis.ipynb, Section 3

# Exploratory Analysis (1/4): Apps Differ Widely

**67% of reviews are 5★ and 17% are 1★; Amazon is the harshest app (54% 1-2★), PhonePe the mildest (10%)**

Written reviews sit 0.0-1.4 stars below each app's public rating, which also counts ratings without a written review. App effect on rating: Cramér's V = 0.12.



## Finding

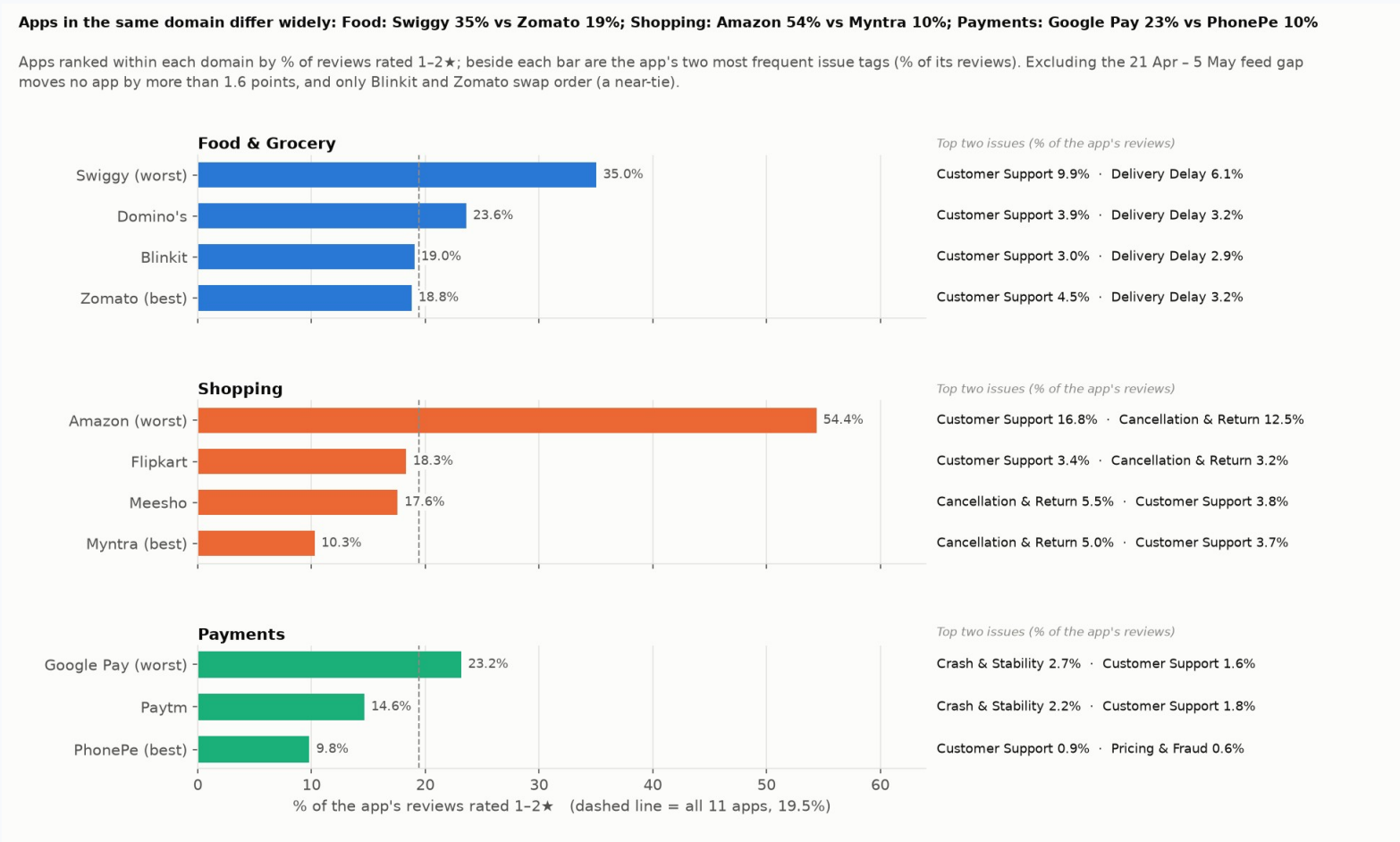
Amazon (54%) and Swiggy (35%) rate 1-2 stars far more often than the other nine apps (10 to 24%). 67% of all reviews are 5 stars and 17% are 1 star. Written reviews sit 0.0 to 1.4 stars below the public rating (Amazon 2.74 vs 4.18).

## Business significance

The gaps between apps are large and lasting, so they reflect real service differences. Compare apps with each other; do not quote absolute satisfaction.

EDA chart 02: rating mix per app, and written reviews against each app's public Play Store rating.

# Exploratory Analysis (2/4): Best and Worst App in Each Domain



## Finding

Food: Swiggy 35.0% vs Zomato 18.8%. Shopping: Amazon 54.4% vs Myntra 10.3%. Payments: Google Pay 23.2% vs PhonePe 9.8%. Swiggy has about twice Zomato's support and delivery complaints; the weaker payment apps lead with crashes.

## Business significance

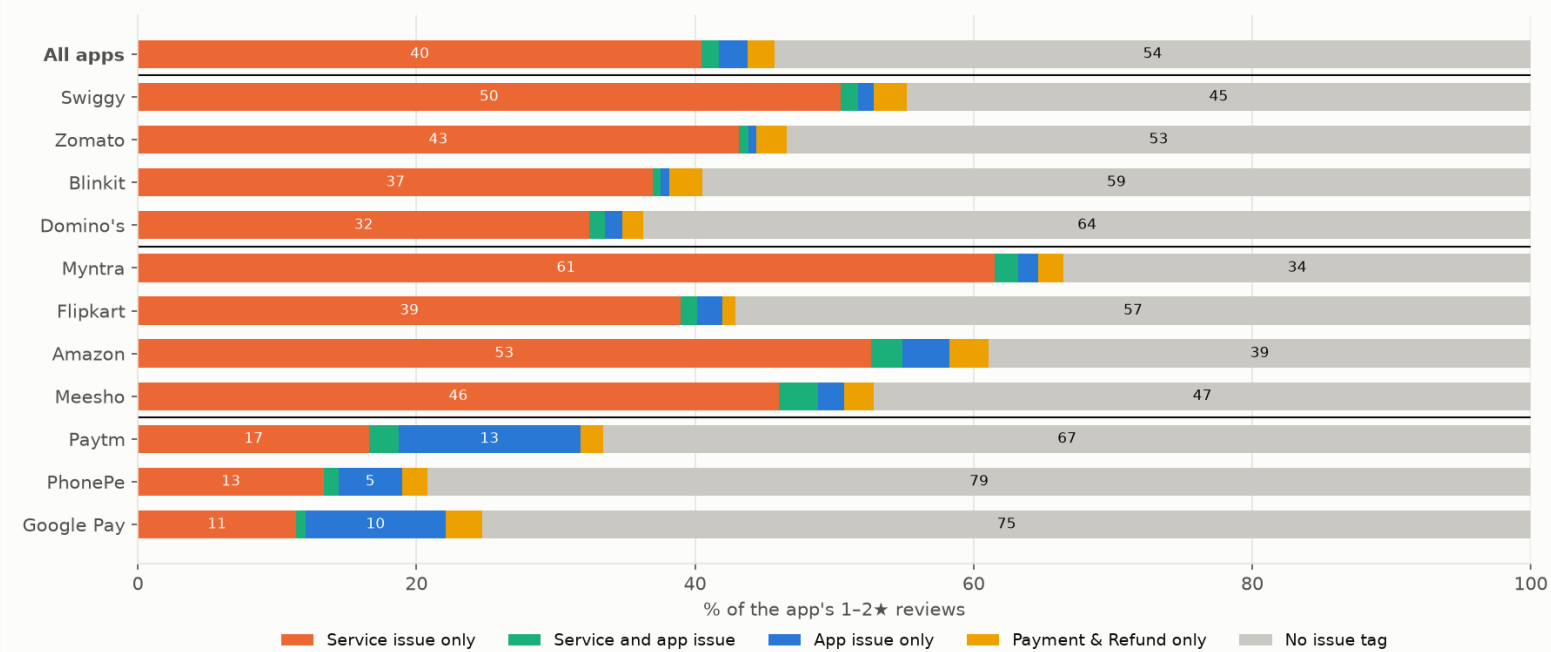
Each team has a different first fix. Blinkit and Zomato are a near-tie. These are written reviews only, not proof of cause.

EDA chart 14: apps ranked inside their domain, with each app's two most frequent issue tags.

# Exploratory Analysis (3/4): Service Problems, Not App Failures

## Among 1-2★ reviews that name an issue, 91% name a service problem and 7% an app problem

What each app's 1-2★ reviews are about, from the keyword tags. Service = delivery, order quality, cancellation/return, support, pricing; app = crash, login/OTP, UI/update. Payment & Refund can be the app, a bank or a gateway, so it is shown separately. 54% of 1-2★ reviews carry no tag, so every share here is a lower bound.



EDA chart 15: what each app's 1-2 star reviews are about, from the keyword issue tags.

### Finding

Among 1-2 star reviews that name an issue, 91% name a service problem (delivery, order quality, returns, support, pricing) and 7% an app problem (crash, login, UI). Payment apps are the exception: app-only tags are 13% at Paytm and 10% at Google Pay.

### Business significance

The classifier finds dissatisfied users, not software failures. Route app complaints to engineering, the rest to operations and support. 54% of 1-2 star reviews carry no tag, so these shares are lower bounds.

# Exploratory Analysis (4/4): A Data Gap, Not Angrier Users

**21 Apr-5 May: in 5 of 11 apps positive reviews fall ~84% while negative reviews fall far less — a feed gap, not an incident**

Affected: Swiggy, Blinkit, Domino's, Flipkart, Amazon. Short positive reviews ('good', 'nice') are what disappear, so median length jumps. Shares such as '% rated 1-2★' are inflated in this window; counts of negative reviews per day are far less affected. The gap is kept in the dataset and flagged; trend, version and July analyses exclude it.

|            | Positive (4-5★) reviews/day, index (before/after avg = 100) |                       |                     | Negative (1-2★) reviews/day, same index |                       |                     | Median review length (words) |                       |                     |
|------------|-------------------------------------------------------------|-----------------------|---------------------|-----------------------------------------|-----------------------|---------------------|------------------------------|-----------------------|---------------------|
|            | Before<br>(8-20 Apr)                                        | Gap<br>(21 Apr-5 May) | After<br>(6-18 May) | Before<br>(8-20 Apr)                    | Gap<br>(21 Apr-5 May) | After<br>(6-18 May) | Before<br>(8-20 Apr)         | Gap<br>(21 Apr-5 May) | After<br>(6-18 May) |
| Swiggy     | 95                                                          | 10                    | 105                 | 94                                      | 78                    | 106                 | 2                            | 18                    | 2                   |
| Zomato     | 117                                                         | 84                    | 83                  | 104                                     | 72                    | 96                  | 2                            | 2                     | 2                   |
| Blinkit    | 89                                                          | 38                    | 111                 | 88                                      | 82                    | 112                 | 2                            | 5                     | 2                   |
| Domino's   | 91                                                          | 16                    | 109                 | 102                                     | 66                    | 98                  | 2                            | 7                     | 2                   |
| Myntra     | 100                                                         | 80                    | 100                 | 114                                     | 87                    | 86                  | 3                            | 3                     | 3                   |
| Flipkart   | 89                                                          | 12                    | 111                 | 91                                      | 63                    | 109                 | 2                            | 5                     | 2                   |
| Amazon     | 102                                                         | 41                    | 98                  | 96                                      | 97                    | 104                 | 4                            | 13                    | 5                   |
| Meesho     | 104                                                         | 95                    | 96                  | 101                                     | 102                   | 99                  | 3                            | 3                     | 3                   |
| Paytm      | 100                                                         | 54                    | 100                 | 71                                      | 79                    | 129                 | 2                            | 2                     | 2                   |
| PhonePe    | 100                                                         | 89                    | 100                 | 102                                     | 90                    | 98                  | 2                            | 2                     | 2                   |
| Google Pay | 80                                                          | 104                   | 120                 | 87                                      | 88                    | 113                 | 3                            | 2                     | 2                   |

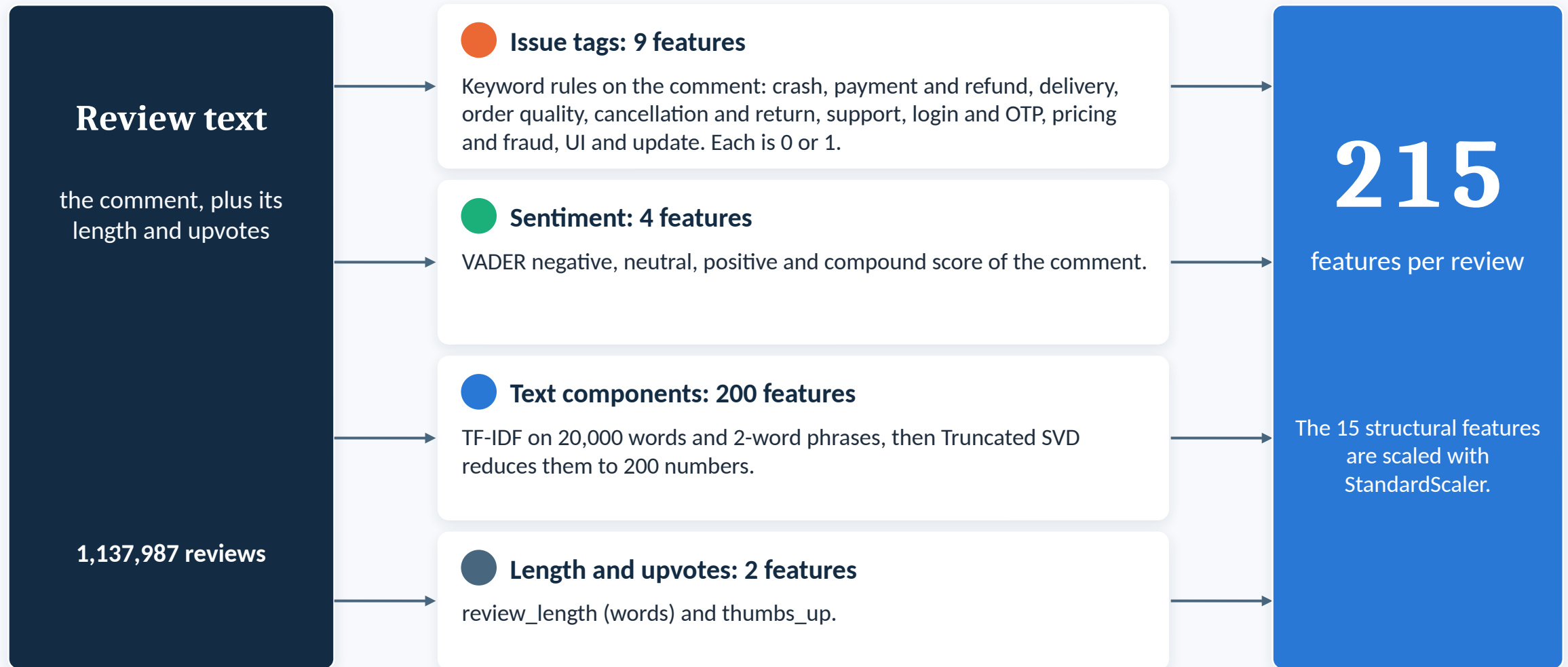
Finding

From 21 April to 5 May positive reviews fall 59 to 90% for Swiggy, Blinkit, Domino's, Flipkart and Amazon, while negative reviews fall only 3 to 37%. Swiggy's weekly share of 1-2 star reviews jumps from about 33% to 87%.

Business significance

A '% negative' dashboard would raise a false alarm. Count negative reviews per day instead. We keep and flag these rows, and exclude them from trend comparisons.

# Feature Engineering: Method and Features Used



# Feature Engineering: Justification and Effect

## Why this method suits the problem

**Compact text:** a 1.1 million x 20,000 sparse matrix is slow and noisy for a Random Forest. SVD gives a small dense matrix.

**No stop-word list:** the standard list removes "not", "no" and "never", so "not good" would look like "good". We keep them.

**Flags and sentiment:** add meaning single words miss, such as whether the review is about a refund and how negative it is.

**Length kept explicitly:** complaints are much longer than praise (14 vs 2 words), and a short review gives the text little to work with.

20,000

text columns before

200

after Truncated SVD

100x

fewer text columns

62.7%

of the TF-IDF variance kept

## Does each group help? (test PR-AUC)

**Text alone:** 0.85

**Sentiment alone:** 0.74 to 0.80

**All 215 features:** 0.871 and 0.873

# Model Development: Task, Models and Data Split

Prediction task: binary classification of `is_problematic` (rated 1 or 2 stars). Regression metrics (R2, MAE, MSE, RMSE) do not apply.

## Baseline

Always predicts "not problematic". Accuracy 80.5% but it finds no complaints.

## Logistic Regression

A fast, interpretable linear model; the coefficients show the direction of each feature.

## Random Forest

150 trees that learn non-linear patterns and give feature importance.

Both models use `class_weight = balanced` (only 19.5% of reviews are problems). Settings: Logistic Regression `C = 1`, `max_iter 1000`; Random Forest 150 trees, each on 30% of the rows, at least 100 reviews per leaf (9.8 MB). One setting per model was checked on a validation split of the training data, never the test set (`C = 1` is best; 25 reviews per leaf adds only 0.004 PR-AUC at 2.8x the size).

## Data split

Random split (stratified, random state 42)

Train 80%: 910,389 reviews

Test 20%: 227,598

Out-of-time test

Train April to August: 1,008,932

1-20 Sep:  
129,055

# Model Evaluation: Results on the Test Set

| Model (test set, 227,598 reviews) | Accuracy | Precision | Recall | F1    | ROC-AUC | PR-AUC |
|-----------------------------------|----------|-----------|--------|-------|---------|--------|
| Majority-class baseline           | 0.805    | 0.000     | 0.000  | 0.000 | 0.500   | 0.195  |
| Logistic Regression               | 0.922    | 0.767     | 0.858  | 0.810 | 0.940   | 0.871  |
| Random Forest                     | 0.914    | 0.737     | 0.870  | 0.798 | 0.941   | 0.873  |

Blue = better of the two models. R2, MAE, MSE and RMSE are regression metrics and do not apply to this classification task.

## Confusion matrix counts (test set)

| Model               | True negative | False positive | False negative | True positive |
|---------------------|---------------|----------------|----------------|---------------|
| Logistic Regression | 171,744       | 11,538         | 6,297          | 38,019        |
| Random Forest       | 169,539       | 13,743         | 5,756          | 38,560        |

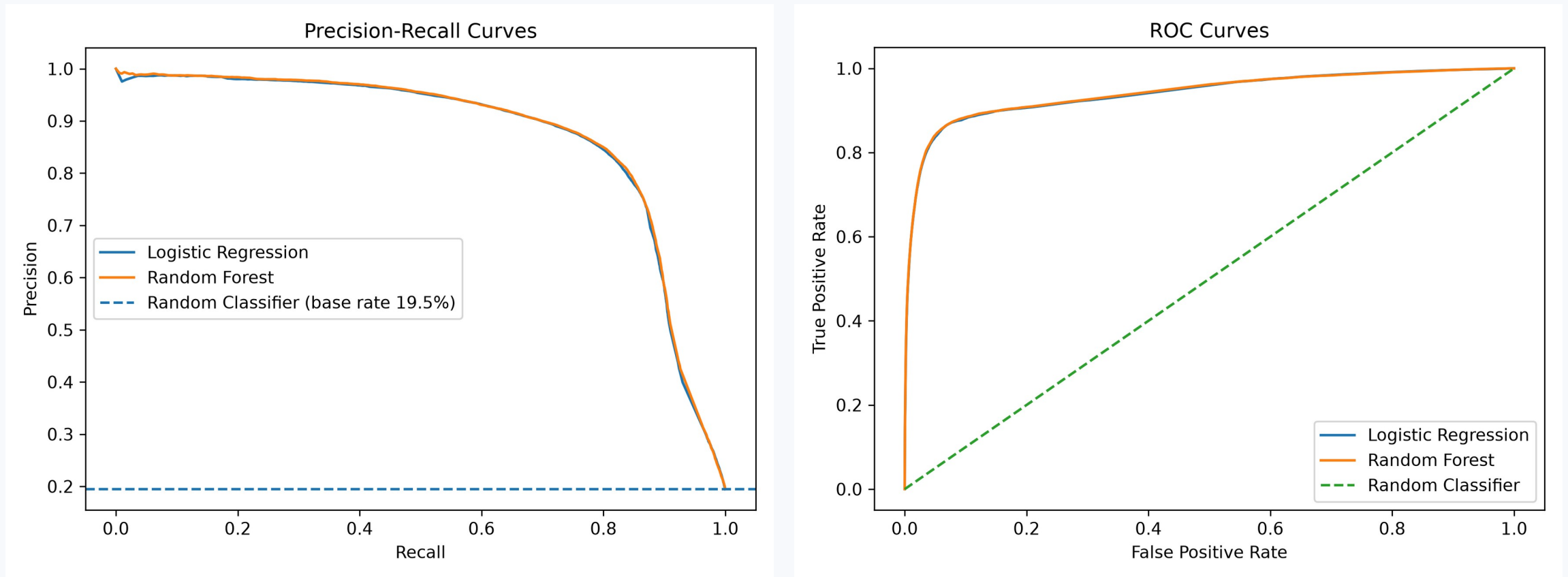
0.87 vs 0.195

PR-AUC of the models against random guessing

87% / 86%

of complaints found (Random Forest / Logistic Regression)

# Model Evaluation: Precision-Recall and ROC Curves



The curves almost overlap: the two models rank reviews alike, and the difference between them is mostly where the 0.5 threshold falls. At 80% recall precision is 0.85 (Random Forest) and 0.84 (Logistic Regression); at 90% recall both are 0.58, about three times the 0.195 base rate.

# Result Interpretation: Strengths and Limitations

## Strengths

- PR-AUC 0.87 is more than four times the 0.195 base rate; accuracy is above 91%.
- A model that never flags anything scores 80.5% but finds no complaints; ours find 86 to 87%.
- Holds on future weeks: train April to August, test 1-20 September, PR-AUC drops by less than 0.01.
- Stable: 95% intervals of about  $\pm 0.003$ ; the chosen settings are best or within 0.005 of the best tried.

## Limitations

- The target is the star rating: the model predicts a low rating, not a confirmed app failure.
- Mis-ratings count as problems (a 1-star "nice product").
- Most reviews are 1 to 3 words, so for them the model mostly reads sentiment.
- Payment apps are hardest (PR-AUC 0.75): rarer, shorter complaints.
- Issue tags are keyword rules; TF-IDF, SVD and the scaler were fitted before the split (they do not use the target).

# Initial Business Findings and Recommendations

## 1 Triage automatically

Random Forest flags about 52,000 of 228,000 test reviews and catches 87% of complaints; about 3 in 4 flags are real.

## 2 Route by cause

91% of tagged complaints are about service. Send app issues to engineering; delivery, returns, refunds and support to operations.

## 3 Fix each app's own issue first

Swiggy: support and delivery. Amazon: support and returns. Google Pay and Paytm: crashes and stability.

## 4 Track counts, not percentages

The late-April gap shows a "% negative" metric can raise a false alarm. Count negative reviews per day.

## 5 Choose the model by cost

Random Forest when missing a complaint is costly; Logistic Regression when reviewer time is the limit. For payment apps, expect about 4 in 10 flags to be false alarms.

# Project Tracking and Progress

**Tool and board link:** GitHub Projects, [github.com/users/Adithya-Monish-Kumar-K/projects/4](https://github.com/users/Adithya-Monish-Kumar-K/projects/4) (repository: [github.com/Adithya-Monish-Kumar-K/ba-capstone-app-reviews](https://github.com/Adithya-Monish-Kumar-K/ba-capstone-app-reviews)). 15 of 15 tasks done.

| Backlog            | To Do                                                       | In Progress | Review/Testing | Completed             |
|--------------------|-------------------------------------------------------------|-------------|----------------|-----------------------|
| Assignees          | Board tasks (issues)                                        | Due         | Status         | Evidence (merged PRs) |
| Aditya             | #53, #54, #56-#58 (5): scraper, collection, cleaning, tests | 29 Sep      | Completed      | PR #52, #99           |
| Akshay, Saketh     | #104-#106, #118-#120 (6): EDA charts, tests, write-up       | 1 Oct       | Completed      | PR #121               |
| Harshini, Kanishka | #107-#110 (4): features, classifiers, evaluation, write-up  | 1 Oct       | Completed      | PR #122-#125          |

| Title                                                                                    | Assignees            | Status    | Linked pull requests | Due Date     |
|------------------------------------------------------------------------------------------|----------------------|-----------|----------------------|--------------|
| 1 Train and evaluate classifiers #108                                                    | d-kanishka and Ha... | Completed | #123                 | Oct 1, 2026  |
| 2 Build feature engineering pipeline #107                                                | d-kanishka and Ha... | Completed | #122                 | Oct 1, 2026  |
| 3 Distinctive uni/bi-grams for 1-2 vs 4-5 star reviews (smoothed odds ratio), an... #118 | akshayk13 and R...   | Completed |                      | Oct 1, 2026  |
| 4 Show which issues matter, how they differ by app, which appear together, an... #120    | akshayk13 and R...   | Completed | #121                 | Oct 1, 2026  |
| 5 Establish what the data can and cannot support before any analysis: time co... #119    | akshayk13 and R...   | Completed |                      | Oct 1, 2026  |
| 6 Build EDA script and core charts #104                                                  | akshayk13 and R...   | Completed |                      | Oct 1, 2026  |
| 7 Add per-app and length breakdowns #105                                                 | akshayk13 and R...   | Completed |                      | Oct 1, 2026  |
| 8 Produce EDA results and writeup #106                                                   | akshayk13 and R...   | Completed |                      | Oct 1, 2026  |
| 9 Add model evaluation charts #109                                                       | d-kanishka and Ha... | Completed | #124                 | Oct 1, 2026  |
| 10 Produce model results and writeup #110                                                | d-kanishka and Ha... | Completed | #125                 | Oct 1, 2026  |
| 11 Build Play Store scraper #53                                                          | Adithya-Monish-K...  | Completed | #99                  | Sep 29, 2026 |
| 12 Collect raw review dataset #54                                                        | Adithya-Monish-K...  | Completed | #99                  | Sep 29, 2026 |
| 13 Filter non-English reviews and derive date/length fields #56                          | Adithya-Monish-K...  | Completed | #99                  | Sep 29, 2026 |
| 14 Produce cleaned review dataset #57                                                    | Adithya-Monish-K...  | Completed | #99                  | Sep 29, 2026 |
| 15 Add data-integrity tests for cleaning #58                                             | Adithya-Monish-K...  | Completed | #99                  | Sep 29, 2026 |

Activity history (pull-request merges)

28 Sep: 12 issues created; data PRs merged (#52, #99).

29 Sep: GitHub Projects board created.

30 Sep: 3 EDA issues added; EDA merged (#121); features, classifiers, evaluation and write-up merged (#122-#125).

# Individual Contributions and Next Steps

| Member / Register number                | Completed tasks                                                                                                     | Evidence                                                      |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Aditya Monish Kumar K CB.SC.U4CSE23103  | Play Store scraper; full NEWEST collection (1.20M reviews, 11 apps); cleaning; dataset documentation; project board | scripts/01-02, apps.py, data_io.py, DATA_SOURCES.md           |
| Regella Krishna Saketh CB.SC.U4CSE23649 | EDA script with Akshay: coverage and bias audit, issue analysis, statistical tests, version analysis                | scripts/05_eda.py, EDA.md, data/charts/eda                    |
| Akshay KS CB.SC.U4CSE23104              | EDA script with Saketh: coverage and bias audit, issue analysis, statistical tests, version analysis                | scripts/05_eda.py, EDA.md, data/charts/eda                    |
| Harshini Vennela CB.SC.U4CSE23455       | With Kanishka: TF-IDF + SVD features; Logistic Regression and Random Forest; evaluation charts                      | scripts/05_feature_engineering.py, 06-13, MODEL_EVALUATION.md |
| Kanishka D CB.SC.U4CSE23155             | With Harshini: TF-IDF + SVD features; Logistic Regression and Random Forest; evaluation charts                      | scripts/05_feature_engineering.py, 06-13, MODEL_EVALUATION.md |

## Pending work and Review 2 plan

**Pending:** none for Review 1.

**Review 2:** text mining + time-series forecasting, interactive dashboard, combined recommendations and report, all on the 1.14M-review dataset.

## Submission links ([github.com/Adithya-Monish-Kumar-K/...](https://github.com/Adithya-Monish-Kumar-K/))

**Notebook:** ba-capstone-app-reviews/blob/master/notebooks/Review\_1\_Analysis.ipynb

**Dataset:** ba-capstone-app-reviews/tree/master/data

**Data documentation:** ba-capstone-app-reviews/blob/master/DATA\_SOURCES.md

**Board:** projects/4 ([github.com/users/...](https://github.com/users/...))

**Summary:** ba-capstone-app-reviews/blob/master/README.md